

CSE-4632

Digital Signal Processing Lab

Lab 3

Lab: **Section 2**

Thursday, 18th June, 2026

Task 1:

Background.

$$y[n] = \sum_{k=0}^{M-1} x[k]h[n-k]$$

The given equation represents the output-side implementation of discrete convolution, where each output sample $y[n]$ is computed independently by sliding the impulse response $h[n]$ across the input signal $x[n]$. For every output index n, the algorithm performs a multiply-and-accumulate operation between the overlapping portions of the input signal and the shifted impulse response.
Your task is to implement this output-side convolution algorithm in the provided notebook and verify the results with the expected output. You are required to implement both "Same" and "Full" convolution modes and compare your implementation with the output of `np.convolve()` for validation.

- **Same Convolution:**
  In this mode, the output signal $y[n]$ has the same length as the input signal $x[n]$. Values outside the valid input boundaries are treated as zero using implicit zero-padding.

$$\text{Output Length}_{\text{same}} = N$$

- **Full Convolution:**
  In this mode, the output signal contains all possible overlaps between the input signal and the impulse response, including partial overlaps at the boundaries.

$$\text{Output Length}_{\text{full}} = N + M - 1$$

  where:
  - N is the length of the input signal
  - M is the length of the impulse response

Tasks:

1. Implement output-side Same convolution.
2. Implement output-side Full convolution.
3. Verify the results using `np.convolve()`.

Task 2:

Background.

$$y[n+m] += x[n] \cdot h[m]$$

The given equation can also be interpreted using the input-side implementation of discrete convolution. In this approach, each input sample $x[n]$ contributes to multiple output samples by distributing scaled versions of the impulse response $h[m]$ across the output signal. The final output is obtained by accumulating the contributions generated from all input samples. Your task is to implement this input-side convolution algorithm in the provided notebook and verify the results with the expected output. You are required to implement both "Same" and "Full" convolution modes and compare your implementation with the output of `np.convolve()` for validation.

- **Same Convolution:**
  The output signal maintains the same length as the input signal.

$$\text{Output Length}_{\text{same}} = N$$

- **Full Convolution:**
  The output signal includes all partial and complete overlaps between the input signal and the impulse response.

$$\text{Output Length}_{\text{full}} = N + M - 1$$

  where:
  - N is the length of the input signal
  - M is the length of the impulse response

Tasks:

1. Implement input-side Same convolution.
2. Implement input-side Full convolution.
3. Verify the results using `np.convolve()`.

Task 3:

Two-dimensional images can be processed using the same discrete convolution principles introduced for one-dimensional signals. In this task, a grayscale (black-and-white) image is represented as a 2D NumPy array, where each row corresponds to a one-dimensional signal across the horizontal axis.
A provided function applies **row-wise convolution**: for every row of the image, a 1D convolution is performed independently in `mode='full'`, and the output is truncated to the original row length. Samples to the left of the first pixel are treated as zero through implicit zero-padding. The columns of the image are not mixed—only horizontal (row) filtering is performed.
This operation is useful for understanding how kernel design directly controls local image transformations. By choosing different 1D kernels, the same convolution engine can implement smoothing, shifting, edge detection, and other row-wise effects.

Tasks:

1. **Provided for you**:
   - A black-and-white test image (`image.png`)
   - The `convolution_2d_rowwise()` function that convolves each row of the image with a 1D kernel
2. **Subtask 1: Moving average kernel**: Design a 1D convolution kernel that averages n adjacent pixels in each row. When applied row-wise, each output sample should be the mean of a length-n window ending at the corresponding column (with zero-padding at the left boundary). Implement `averaging_kernel(n)` in the notebook.
3. **Subtask 2: Row shift kernel**: Design a 1D convolution kernel that shifts each row n units to the **right**. Pixels shifted past the right boundary are discarded; positions vacated on the left are filled with zero. Implement `shift_kernel(n)` in the notebook.
4. **Verification**: For both subtasks, display side-by-side plots of the original image and the filtered result. Experiment with at least two different values of n for each subtask and briefly comment on the visual effect.